

Reinforcement Learning

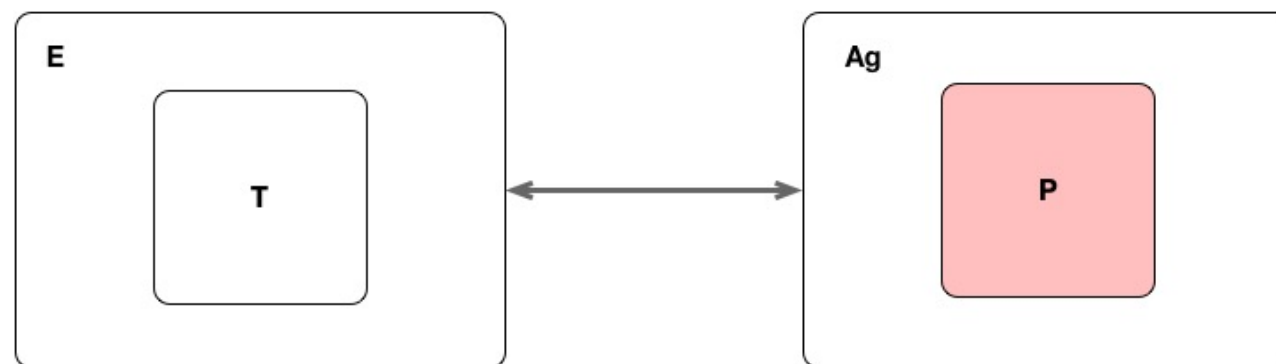
Overview, Q-Learning, Deep Q-Learning

Refresh

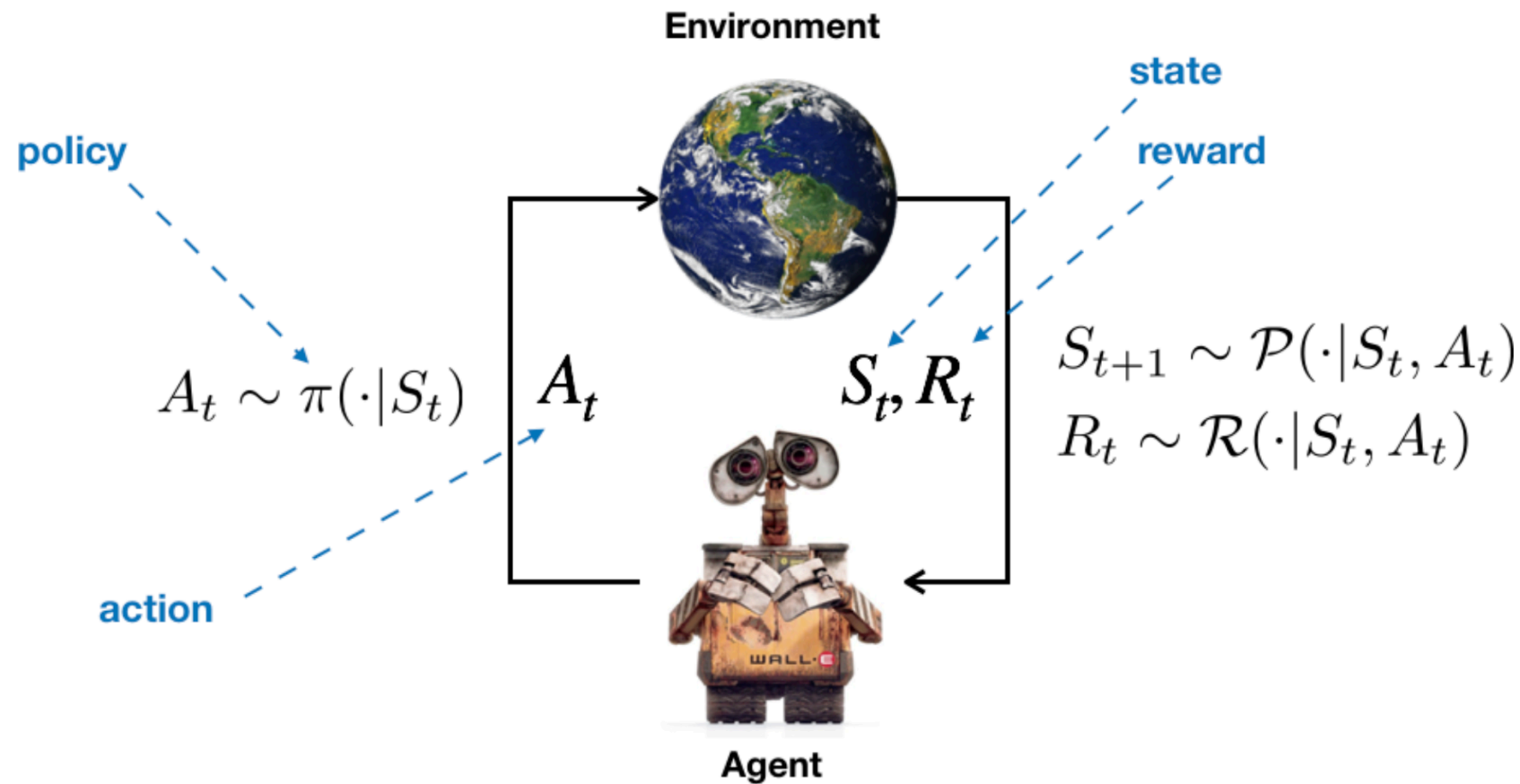
Machine Learning (ML)

A computer program is said to **learn** from **experience** E with respect to some class of tasks T and performance measure P if its performance at tasks in T , as measured by P , improves with experience E .

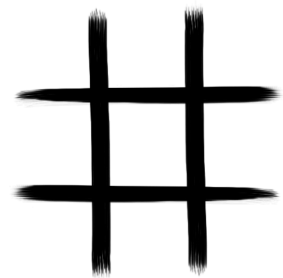
Tom Mitchell, 1997



Reinforcement Learning

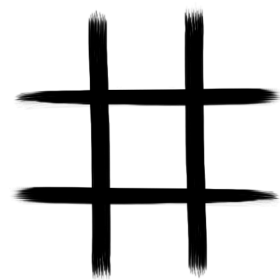


Reinforcement Learning: Quick Example

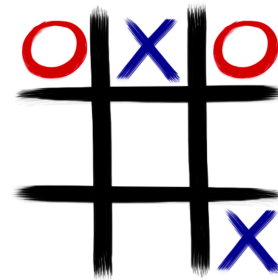


Environment

Reinforcement Learning: Quick Example

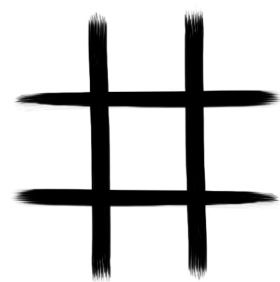


Environment

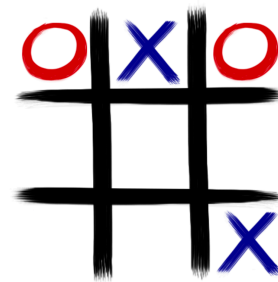


Current State

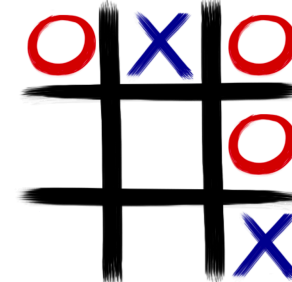
Reinforcement Learning: Quick Example



Environment

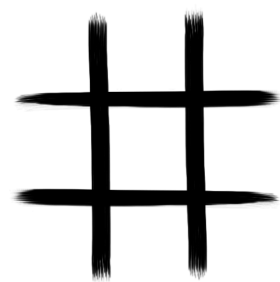


Current State

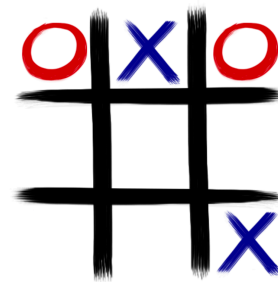


Action

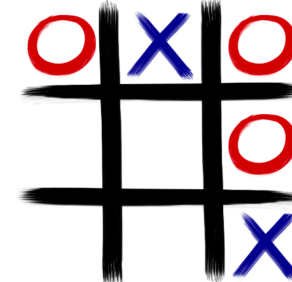
Reinforcement Learning: Quick Example



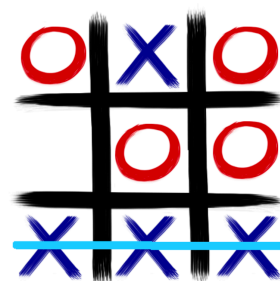
Environment



Current State



Action



Reward

-1

Overview

Reinforcement Learning

- Reinforcement learning is a type of machine learning where an agent learns to make decisions by performing actions in an environment to maximize cumulative rewards.

History

- 1950s: The formal theoretical framework for optimal control is developed by **Richard Bellman**, introducing **dynamical system's state** and **value** functions, core to **reinforcement learning**. The term optimal control emerges, along with **Bellman** equation.

Bellman Equation

$$V(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right)$$

$V(s)$ represents the value of being in state s

$\max_a$ maximum over all possible actions a

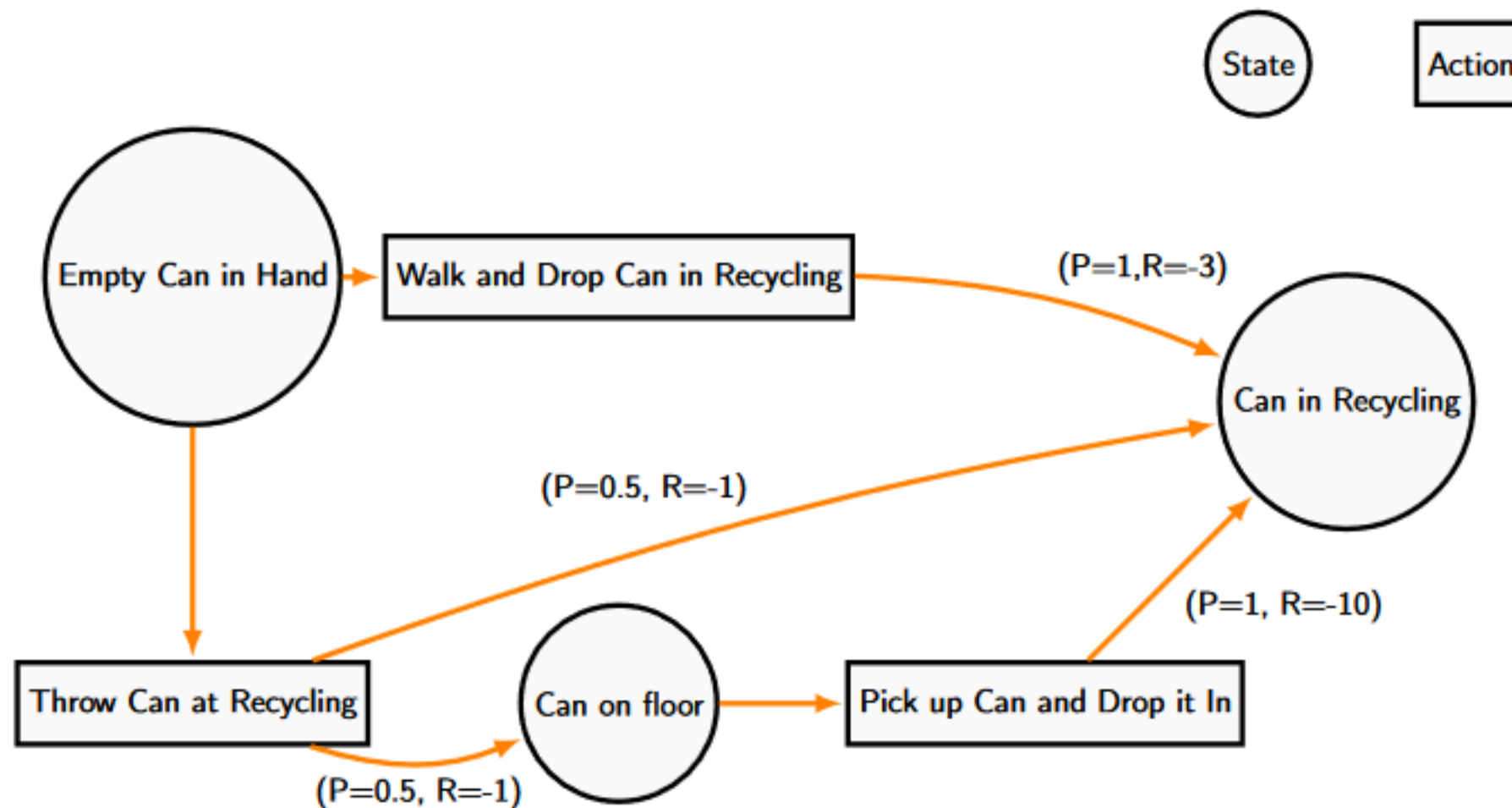
$R(s, a)$ reward of taking action a in state s

γ discount factor that weights the importance of future rewards

$P(s' | s, a)$ probability of transitioning to state s' from state s after action a

In the context of Markov decision processes (MDPs)

Markov Decision Process



History

- 1960s: Ronald Howard enhances **Markovian decision processes** (MDPs) and the policy iteration method, foundational elements in modern reinforcement learning.
- 1970s: Connections between optimal control, dynamic programming, and learning begin to form, with significant figures like Paul Werbos advocating for integrating **dynamic programming** with **learning** methods.

History

- 1980s: Chris Watkins develops the reinforcement learning using the MDP formalism, which is widely adopted, marking a pivotal moment in **formalising reinforcement learning as a field**. The convergence of various threads, including trial-and-error learning and temporal-difference methods, shapes the modern field of reinforcement learning.
- 1990s: Further development of reinforcement learning occur, highlighted by the success of Gerry Tesauro's backgammon playing program, TD-Gammon, which significantly impacts the field.

History

- 1980s: Chris Watkins develops the reinforcement learning using the MDP formalism, which is widely adopted, marking a pivotal moment in **formalising reinforcement learning as a field**. The convergence of various threads, including trial-and-error learning and temporal-difference methods, shapes the modern field of reinforcement learning.
- 1990s: Further development of reinforcement learning occur, highlighted by the success of Gerry Tesauro's backgammon playing program, TD-Gammon, which significantly impacts the field.

History

- 2000s: Integration of reinforcement learning with deep learning begins, setting the stage for more complex models and broader applications in fields such as robotics and automation. It becomes more mainstream in academia and industry, leading to its adoption in areas like automated trading systems and energy management.

History

- 2010s: Breakthrough with DeepMind's development of Deep Q-Networks (DQN) that successfully trained agents to play Atari games at superhuman levels, showcasing the potential of deep reinforcement learning.
- Mid-2010s: AlphaGo's victory over world champion Lee Sedol in Go dramatically demonstrates the capabilities of deep reinforcement learning, drawing significant public and scientific attention. Reinforcement learning is applied more effectively in real-world robotics

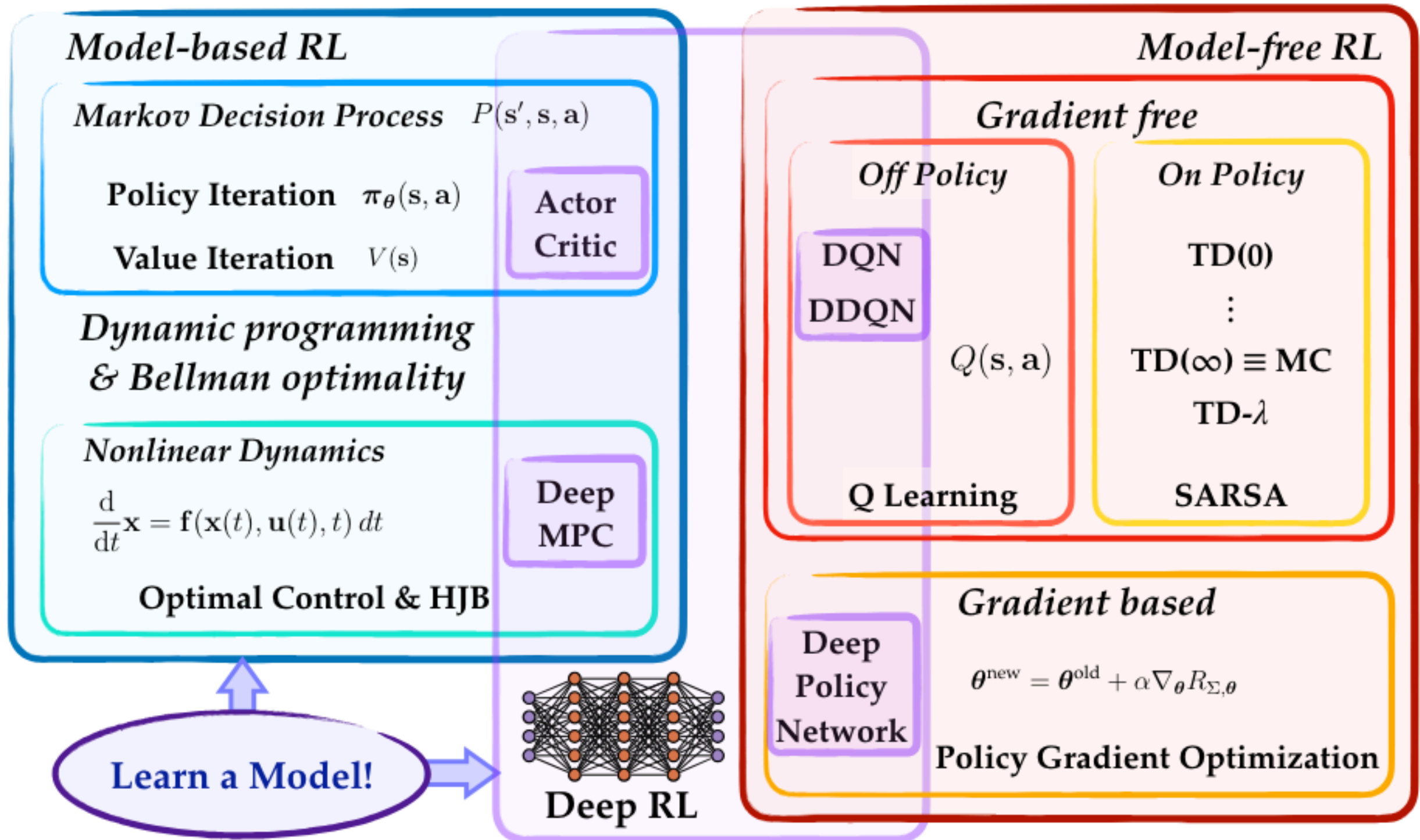


History

- 2020s: Focus shifts towards making RL algorithms more **sample-efficient** and integrating RL with other machine learning paradigms to enhance capability and efficiency. RL with Human Feedback gains attention for LLMs and multimodal models.



The Big Picture



Model and Model Free

Core Aspects

- Model based RL:
 - Constructs and utilizes a model of the environment to simulate outcomes and inform decision-making.
- Model Free:
 - Learns policies or value functions directly from interactions without an explicit model of the environment.

Model and Model Free

Data efficiency and Computation

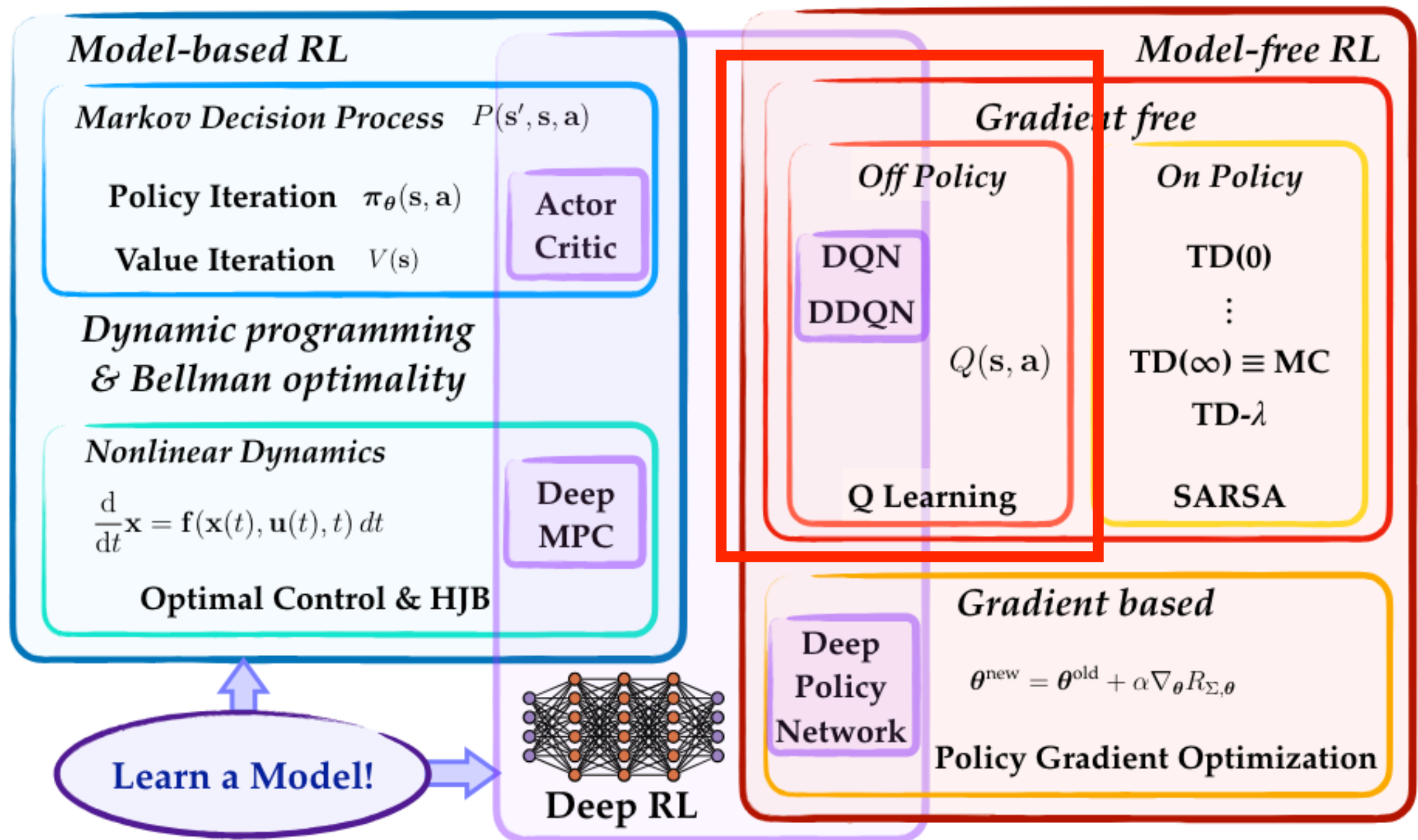
- Model based RL:
 - Model helps simulating next future states, **few interactions**.
 - Probably **more computational expensive** for simulation and planning process
- Model Free:
 - **More interactions to learn**, depends on experience with the environment.
 - More interactions **but simpler ones**

Model and Model Free

Robustness and Flexibility

- Model based RL:
 - **Relies** on the accuracy and robustness of the **model**
- Model Free:
 - More **robust** since it learns with **real data from the environment**

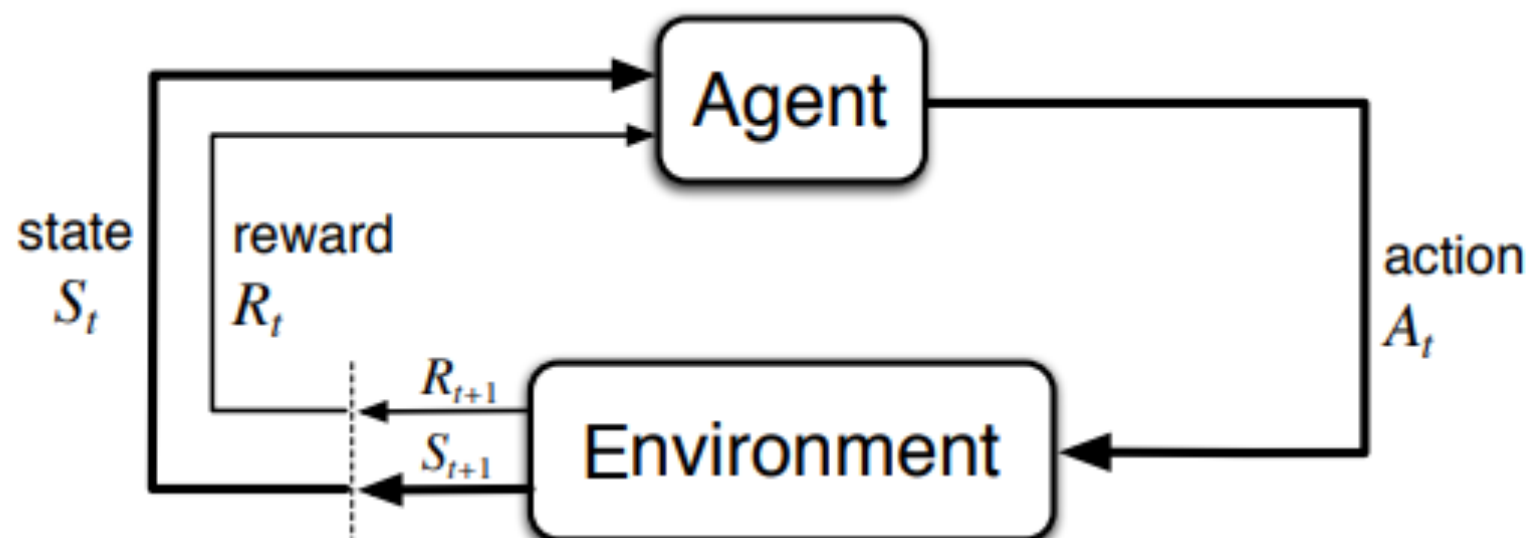
The Big Picture



Q-Learning

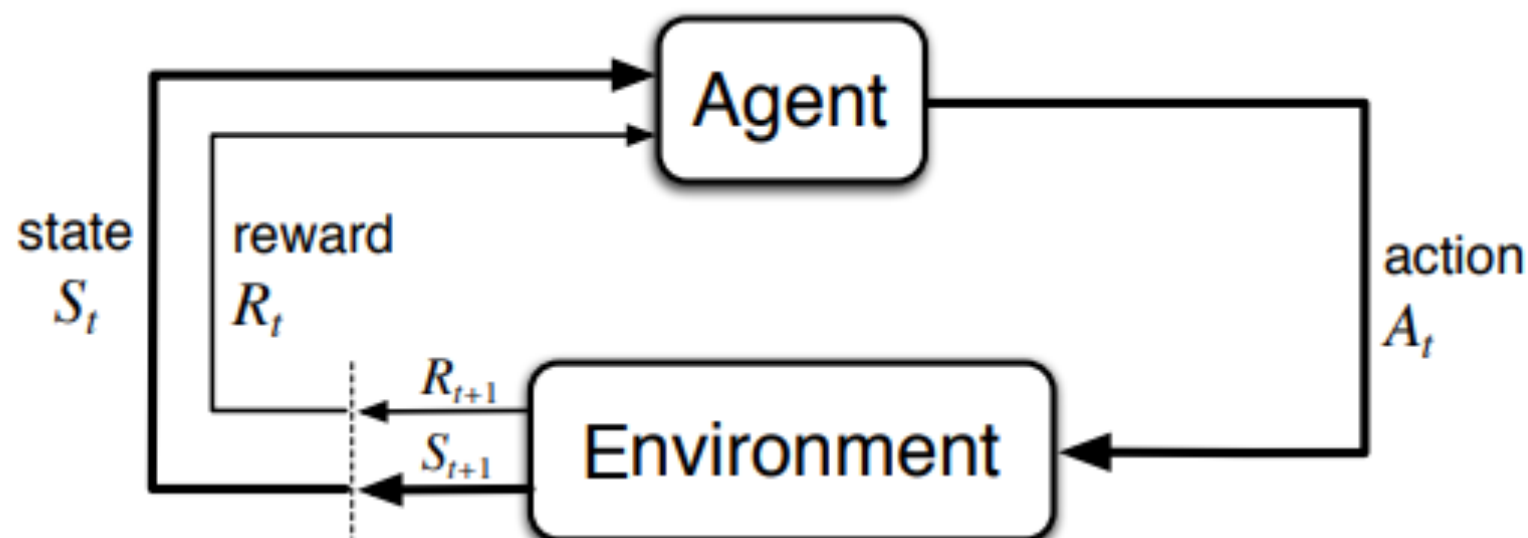
Q-Learning

- State s - representation of the environment at a given time
- Action a - a decision made from a given state s
- Reward r - a signal from the environment in response to the agent's action indicating its immediate value.



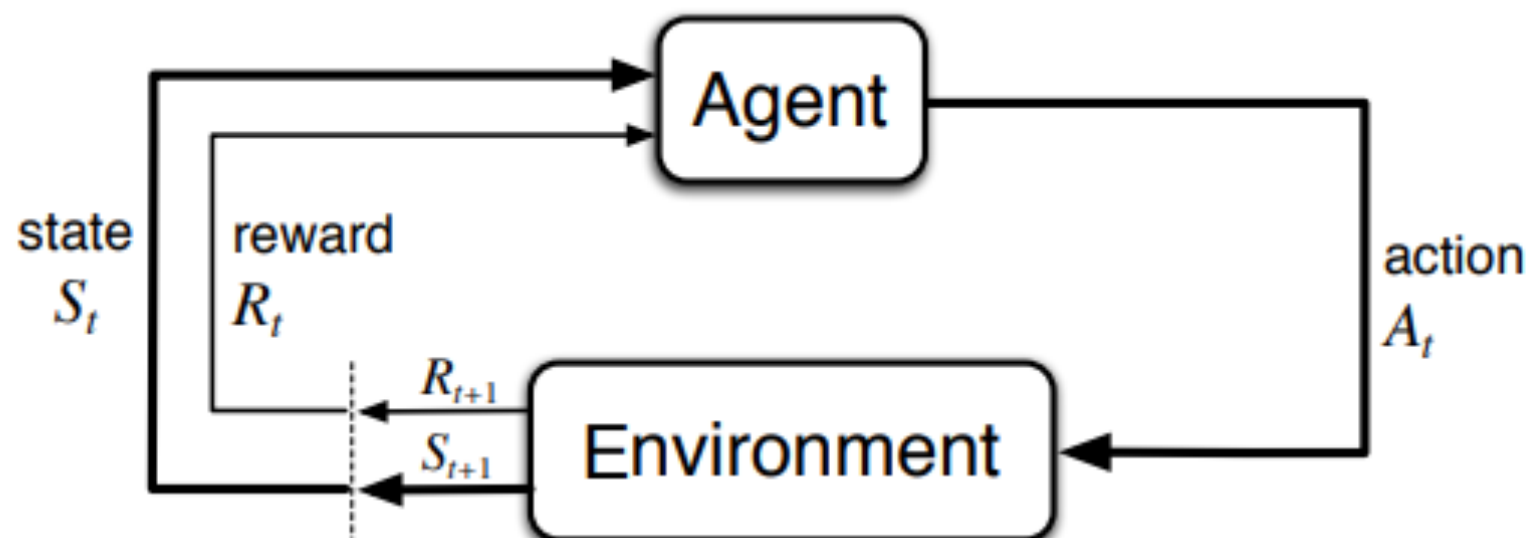
Q-Learning

- Policy π - strategy the agent employs to pick its action based on the current state s .
- Value Function V - estimates how good is being in state s
- Action-Value Function Q : Estimates the value of taking a specific action in a given state.



Q-Learning

- Operation can be episodic or continues
 - **Episodic**: agent resets the environment after
 - **Continuos**: continuos interaction, no reset.

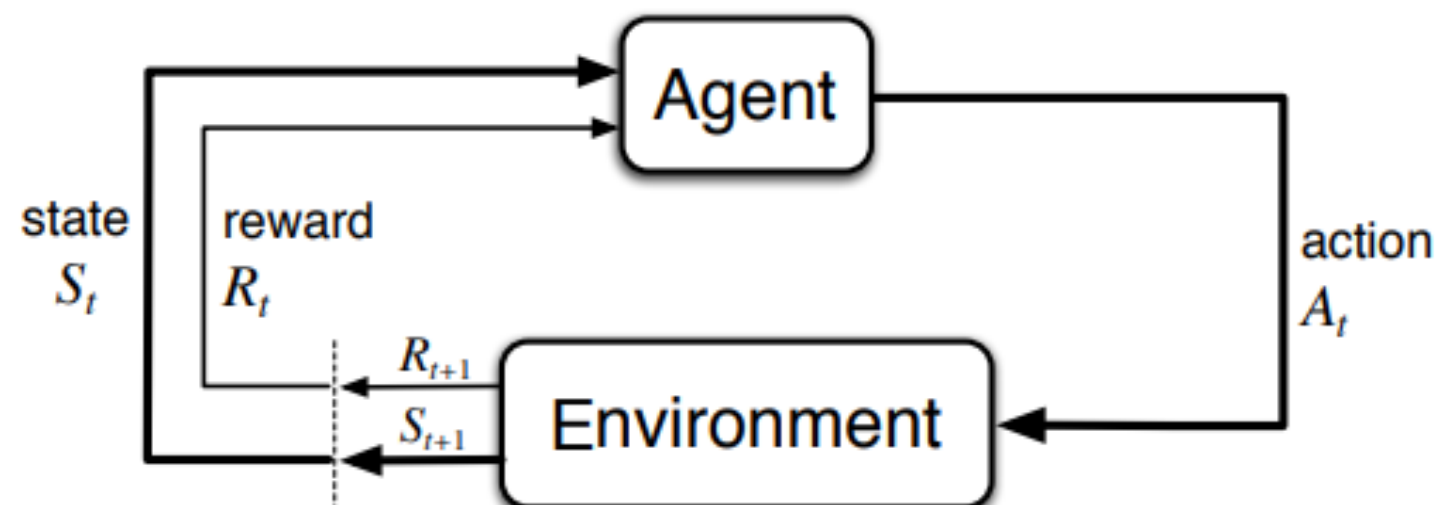


Q-Learning

- Q-learning is a model-free reinforcement learning technique used to **learn** the **value of an action** in a particular state. It uses a **Q-table** or a **Q-function** to map state-action pairs to values (rewards).

Q-Learning

- Off-policy: behaviour (behaviour policy) is different from the policy that is evaluated and improved (target policy).
- On-policy: policy used to generate the behaviour is also used to learn from it. Learning process directly accounts policy's own biases and exploration strategies.



Q-Learning

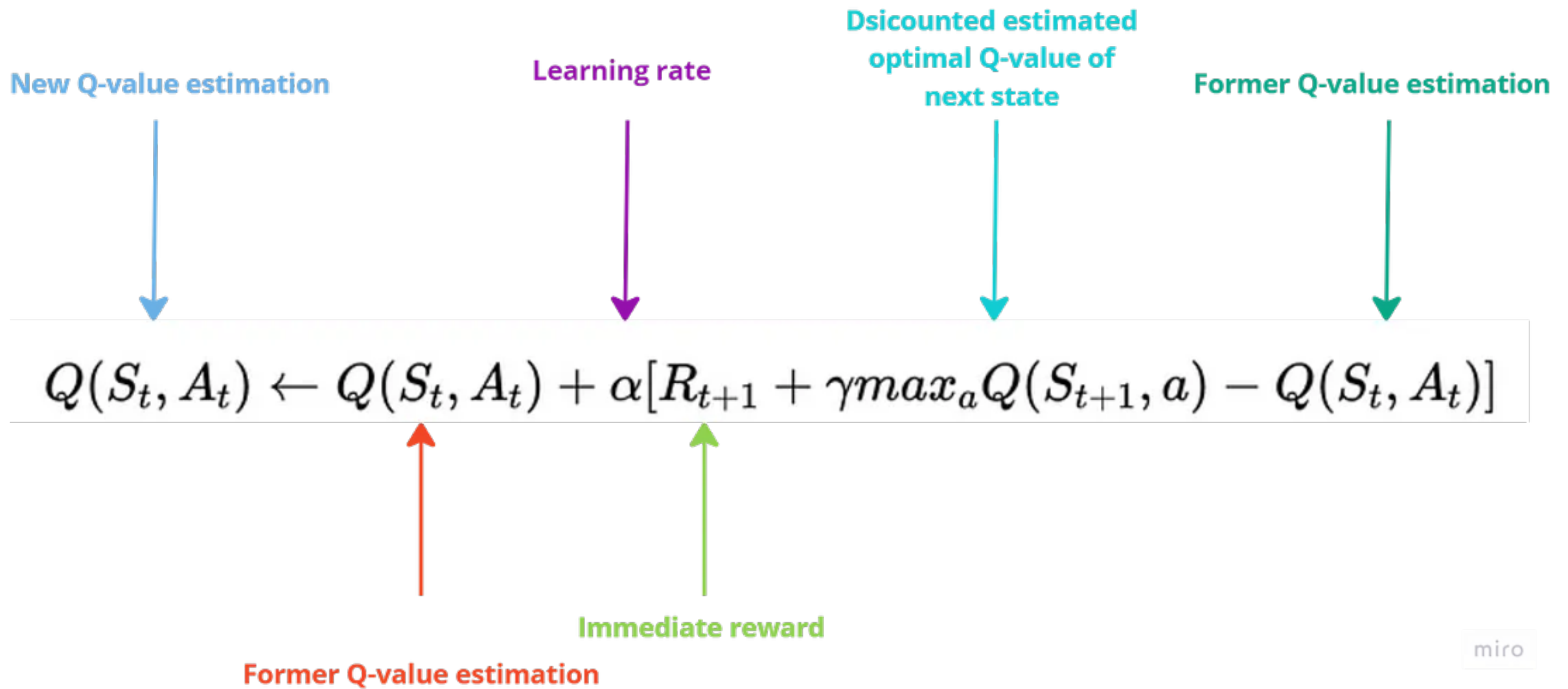
- Q Table is a matrix with s states by a actions
 - Each element holds the value of a given s



Q-Learning Algorithm

1. Initialize the Q-values $Q(s, a)$ arbitrarily for all state-action pairs.
2. Loop for each episode:
 1. Initialize the state s
 2. **Repeat** for each step of the episode:
 1. Choose action a from state s using a **policy** derived from Q
 2. Take action a , observe the rewards r , and next state s'
 3. Update the Q-Value:
 - $$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$
 4. $s \leftarrow s'$
 3. **Until** episode ends

Q-Value function



Policy?

- Usually is a greedy approach (ϵ -greedy)
 - $\pi(s) = \arg \max_a Q(s, a)$
 - in each state, choose the action of highest learned Q value
 - $\pi(s) = \begin{cases} \text{random action,} & \text{with probability } \epsilon \\ \arg \max_a Q(s, a), & \text{with probability } 1 - \epsilon \end{cases}$
- Summary: Q-learning learns the optimal policy indirectly by learning Q-values, without needing a model of the environment.

Question 2 - Solve it 10 points

An agent operates in a discrete two-dimensional grid world composed of unit cells, with a total size of 3×3 . The environment is defined using Cartesian coordinates (x, y) , where the origin $(0, 0)$ is located at the bottom-left corner of the grid and the coordinates increase as in a standard 2D Cartesian map (i.e., x increases to the right and y increases upwards). The agent starts at position $A = (0, 0)$ and the objective is to reach the goal position $B = (2, 2)$. The agent can move only one cell at a time using the actions *Up*, *Down*, *Left*, or *Right*. Diagonal movements are not allowed. Any action that would move the agent outside the grid terminates the simulation immediately. A Q-learning algorithm has been applied and produced a Q-table containing values for each state-action pair. The agent follows a greedy policy, selecting the action with the highest Q-value in the current state. Your task is to **choose the Q-table that allows the agent to move from A to B**. **Note:** If multiple actions have the same Q-value, the agent selects the action that appears in the leftmost column of the table.

	↑	↓	←	→
(0,0)	0.7	0.5	0.7	0.8
(0,1)	1	0.8	0.1	0.2
(0,2)	0.8	1	1.0	0.0
(1,0)	1	0.5	0.5	0.3
(1,1)	0.1	0.1	0.7	0.8
(1,2)	0.8	1	0.4	0.3
(2,0)	1	0.5	0.3	0.8
(2,1)	0.8	1	0.6	0.1
(2,2)	0.8	1	0.8	0.9

a)

	↑	↓	←	→
(0,0)	0.2	0.5	0.1	0.7
(0,1)	0.8	1	0.5	0.1
(0,2)	0.8	1	0.2	0.9
(1,0)	1	0.5	0.9	0.5
(1,1)	0.1	0.1	1.0	0.8
(1,2)	0.8	0.2	0.7	1.0
(2,0)	1	0.5	0.2	0.2
(2,1)	0.8	1	0.7	0.7
(2,2)	0.8	0.7	0.9	0.4

b)

	↑	↓	←	→
(0,0)	1	0.8	0.1	0.5
(0,1)	0.8	1	0.1	0.8
(0,2)	0.8	1	0.5	0.8
(1,0)	0.2	0.5	0.6	0.7
(1,1)	0.1	0.1	0.2	0.8
(1,2)	0.7	0.5	0.4	0.6
(2,0)	1	0.5	0.4	0.0
(2,1)	0.4	0.9	1.0	0.5
(2,2)	0.8	0.7	0.4	0.2

c)

d) None of the above.

Question 3 - Solve 15 points

Consider a Q-learning agent with three states: S_1 , S_2 , and S_3 , and four possible actions per state: A_1 , A_2 , A_3 , and A_4 . The agent's current Q-table is:

State	$Q(S, A_1)$	$Q(S, A_2)$	$Q(S, A_3)$	$Q(S, A_4)$
S_1	2.0	1.0	2.0	0.5
S_2	0.0	1.5	1.0	1.0
S_3	3.0	2.5	3.0	2.0

The agent follows an ε -greedy policy with $\varepsilon = 0.10$. If there's a tie in greedy mode, the agent chooses randomly among the actions with the highest Q value. The agent is currently in state S_3 . What is the probability that it chooses action A_1 in this state?

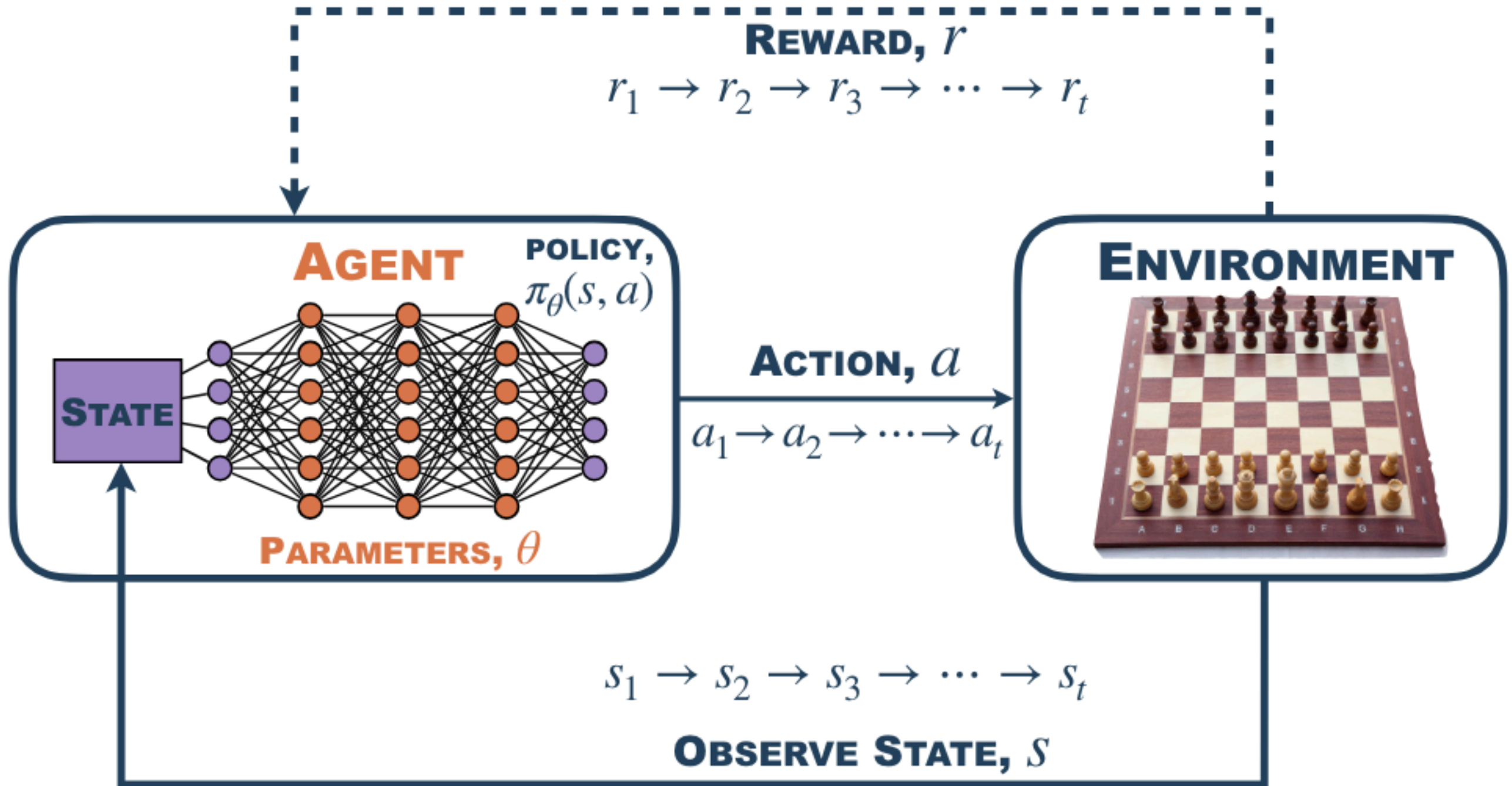
- a) 0.50
- b) 0.25
- c) 0.45
- d) None of the above

Answer:

Policy ?

- One of the simplest and most used in Q-learning is the ϵ -greedy policy
- According to a parameter ϵ next move could be random choice or based on the Q value of the table
 - ϵ decreases along time. (hmmm sounds familiar?)
- Balances Exploration vs Exploitation

Deep Q Learning



Deep Q Learning

- Deep Q-Learning uses a neural network to approximate the Q-value function. The network, often referred to as a Deep Q-Network (DQN), **inputs the state** of the environment and outputs a **vector of action values**.

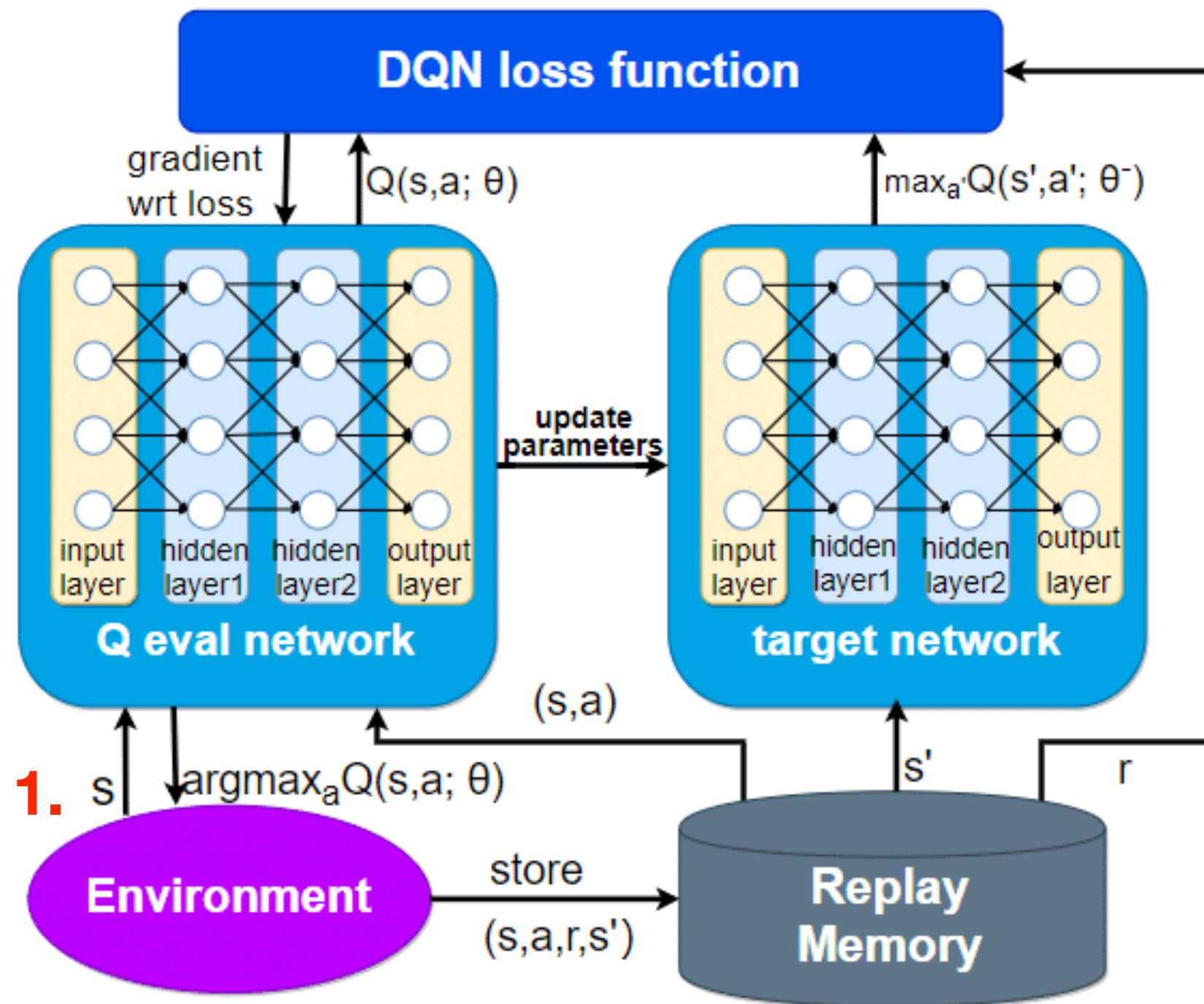
Deep Q Networks

- Training process: interacting repeatedly with the environment and updating the policy based on sampled experiences. Introduces the concept of **memory** of states (replay) to update the weights of the networks.

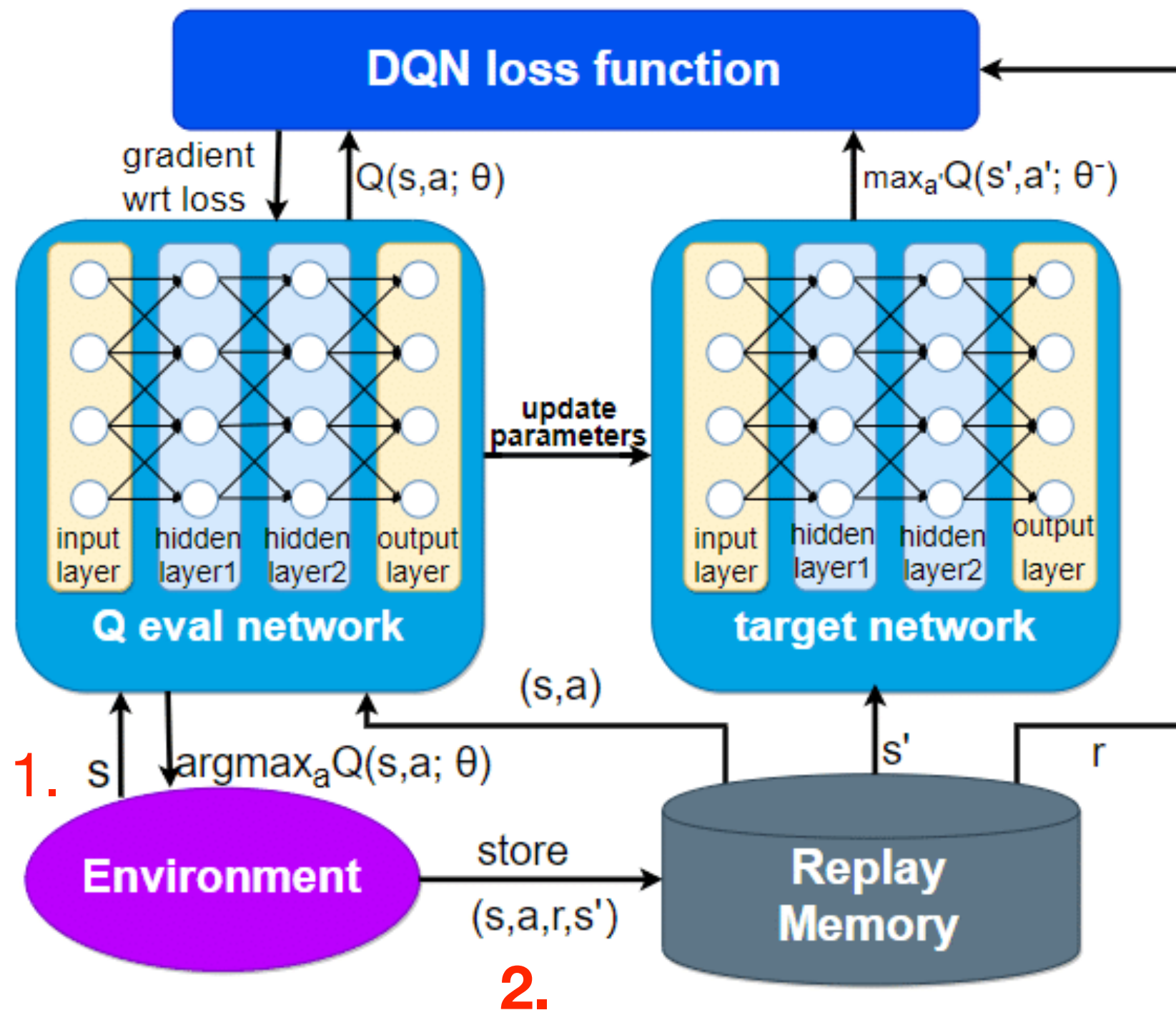
Deep Q Networks

- Define two networks: **policy** and **target** network.
1. Agent starts in initial state s and selects an action a based on a policy, which can come from the **policy** network.
 2. After action a , the agent observes the reward r and the new state s' .
 3. Transition (s, a, r, s') is stored in **replay memory**.
 4. Periodically, a batch of transitions is randomly sampled from the replay memory; **the policy (Q eval) network's weights** are updated via gradient descent on a loss function that measures the difference between the current and target Q-values:
$$\text{Loss} = \left(r + \gamma \max_{a'} Q_{\text{target}}(s', a') - Q(s, a) \right)^2$$
 5. Every few steps, the weights of the **target network** are updated to match those of the **policy network** (network sync).

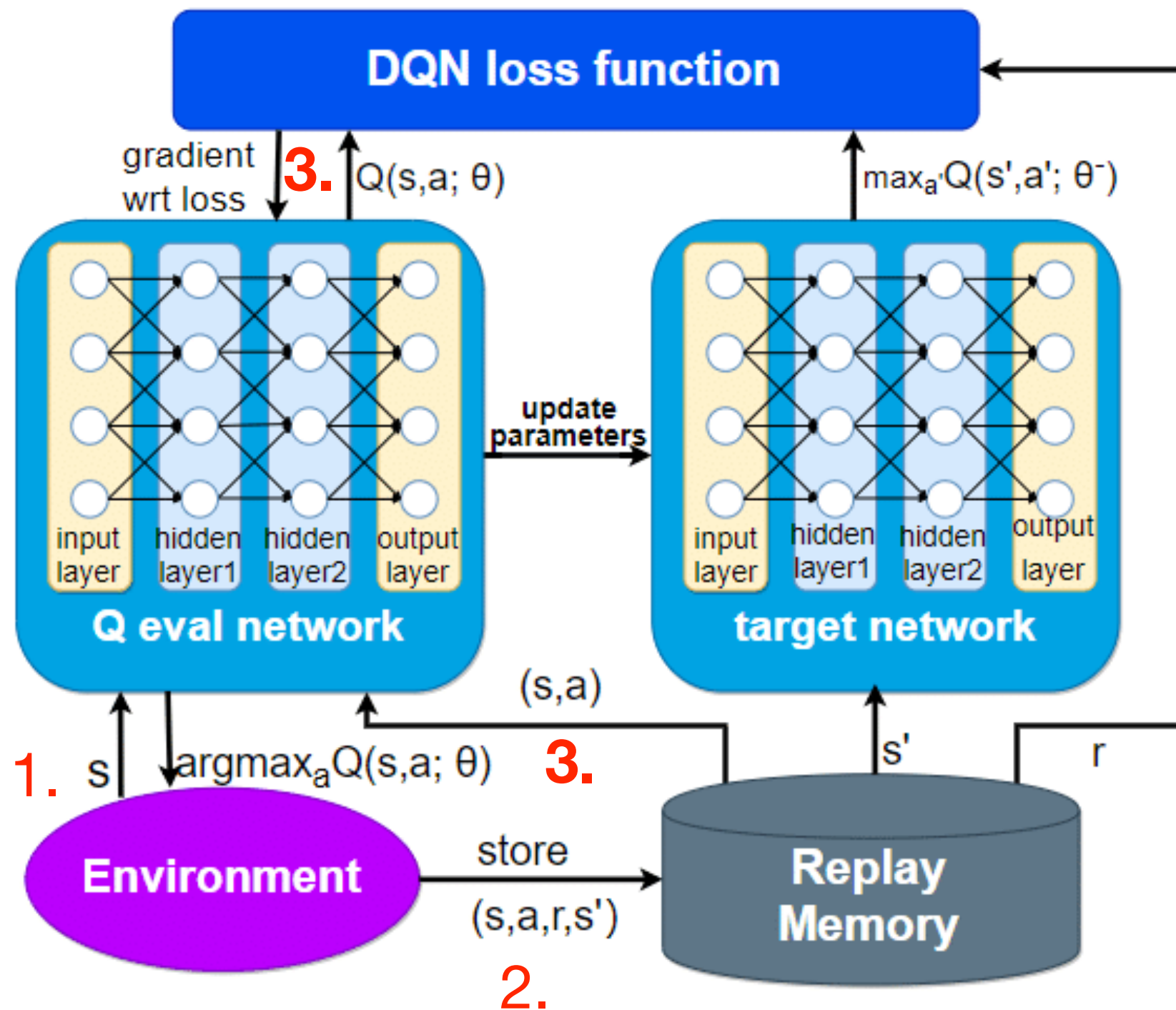
Deep Q Networks



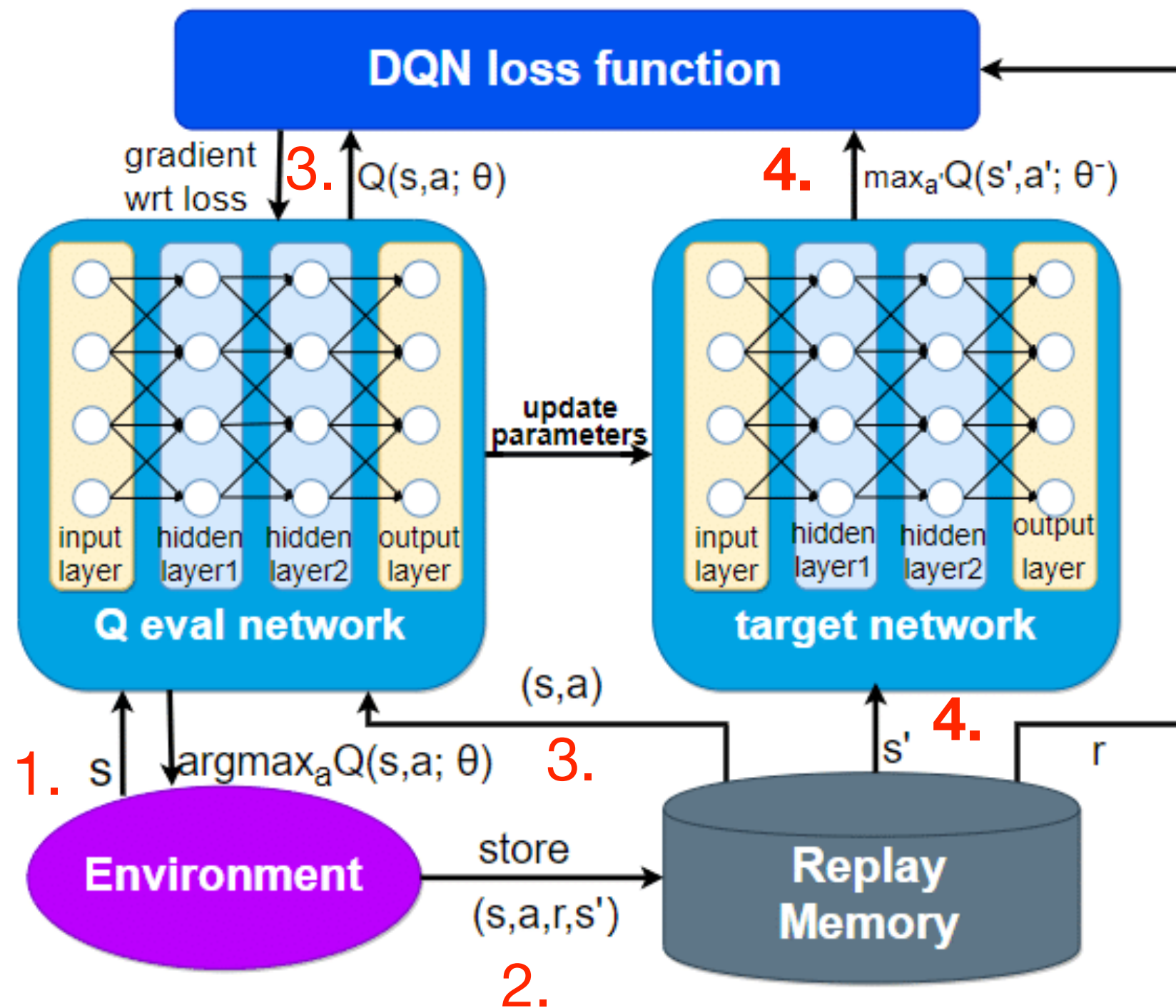
Deep Q Networks



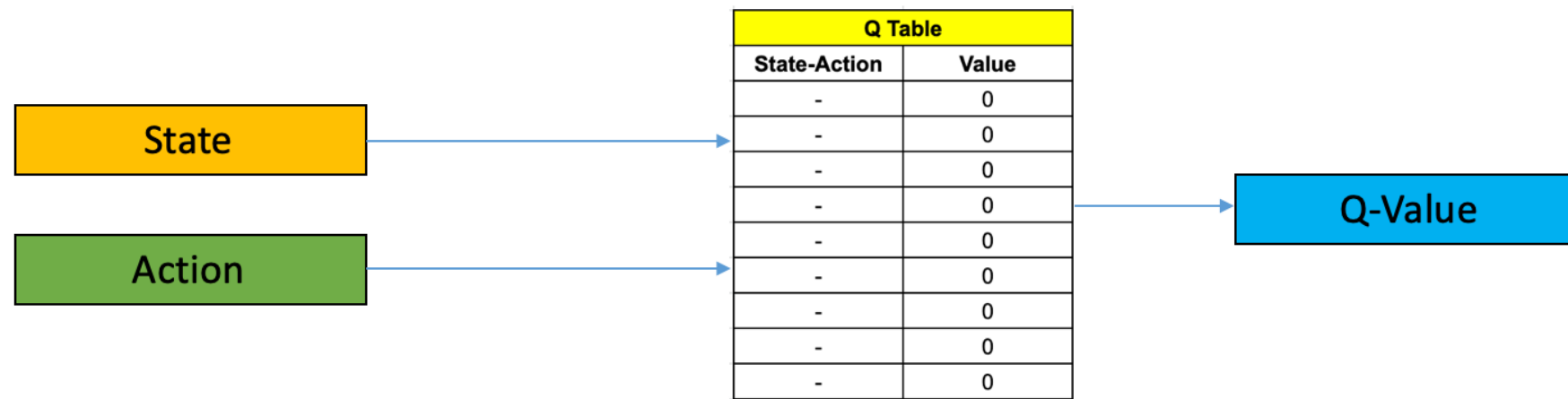
Deep Q Networks



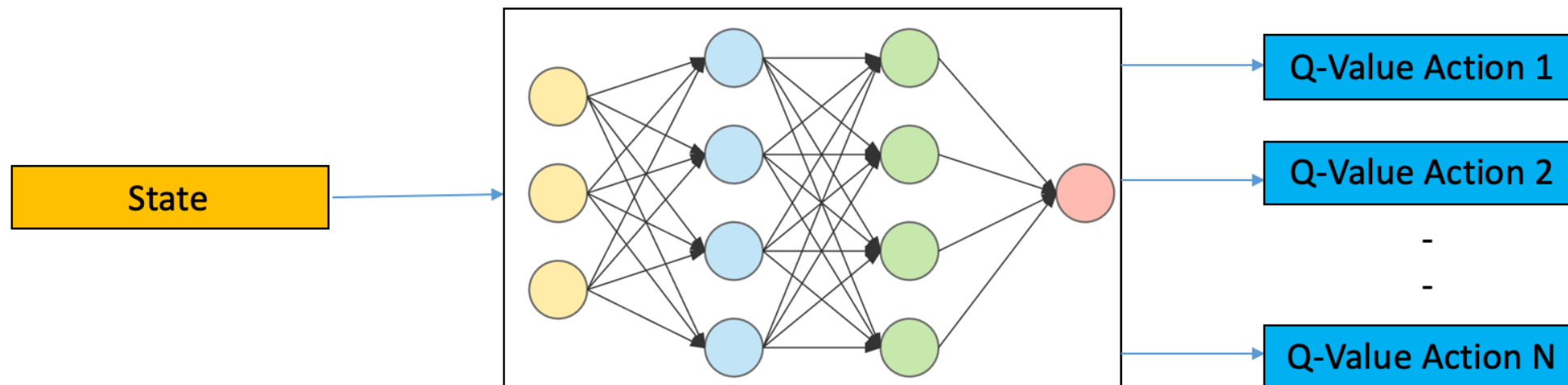
Deep Q Networks



Summary



Q Learning



Deep Q Learning

References

- Christopher M. Bishop and Hugh Bishop, Deep Learning - Foundations and Concepts (2024). Springer 2024,
- Prince, S. J. (2023). Understanding Deep Learning. MIT Press.
- Drori, I. (2022). The Science of Deep Learning. Cambridge University Press.
- Brunton, S. And Kutz, J. N. (2017). Data Driven Science & Engineering - Machine Learning, Dynamical Systems, and Control
- <https://www.youtube.com/watch?v=1kV-rZZw50Q>
- <https://www.youtube.com/watch?v=Mz7Qp73lj9o>
- <https://www.youtube.com/watch?v=xiChGsduWEU>
- <https://www.youtube.com/watch?v=WMmGzx-jWvs>